# Generics, Collections, Functional Interfaces & Lambdas

# Generics, Collections Framework, Functional Interfaces & Lambda Expressions — Notes

## 1. Generics in Java

- **Generics** allow classes, interfaces, and methods to operate on **types specified as parameters**.
- Enables **type safety** at compile time and removes the need for explicit casting.
- Introduced in **Java 5**.

### Without Generics (old way)

```
List list = new ArrayList();
list.add("Hello");
list.add(10);                    // no error at compile time - mixed types allowed!
String s = (String) list.get(0);  // explicit casting needed
```

### With Generics

```
List<String> list = new ArrayList<>();
list.add("Hello");
// list.add(10);                 // COMPILE-TIME ERROR - type safety enforced
String s = list.get(0);          // no casting needed
```

### Generic Class

```
class Box<T> {              // T = Type placeholder
    private T value;

    void set(T value) { this.value = value; }
    T get() { return value; }
}

Box<String> stringBox = new Box<>();
stringBox.set("Hello");

Box<Integer> intBox = new Box<>();
intBox.set(100);
```

### Generic Method

```
<T> void printArray(T[] arr) {
    for (T item : arr) {
        System.out.println(item);
    }
}
```

### Common Type Parameter Names

| Symbol | Meaning |
| --- | --- |
| `T` | Type |

| | |
|---|---|
| `E` | Element (used in collections) |
| `K` | Key |
| `V` | Value |
| `N` | Number |

### Bounded Type Parameters

Restrict the types that can be used.

```
class Calculator<T extends Number> {   // only Number and its subclasses allowed
    T num;
    Calculator(T num) { this.num = num; }

    double square() {
        return num.doubleValue() * num.doubleValue();
    }
}
```

### Wildcards ( `?` )

Used when the exact type is unknown.

| Wildcard | Meaning |
|---|---|
| `<?>` | Unbounded — any type |
| `<? extends T>` | Upper bounded — T or its subclasses (read-only, “producer”) |
| `<? super T>` | Lower bounded — T or its superclasses (“consumer”) |

```
void printList(List<?> list) {              // accepts List of any type
    for (Object o : list) System.out.println(o);
}

void addNumbers(List<? super Integer> list) { // can add Integer or its supertype
    list.add(10);
}
```

**Benefits of Generics:** - **Type Safety** – errors caught at compile time, not runtime - **No Casting** – cleaner code - **Code Reusability** – same class/method works with different types

**One-liner:** *“Generics provide compile-time type safety and eliminate the need for casting by letting classes/methods work with a parameterized type.”*

---

## 2. Overview of Collection Framework

The **Collection Framework** is a unified architecture (set of classes and interfaces) for storing and manipulating groups of objects (lists, sets, queues, maps).

### 2.1 Architecture Diagram

```
                  Iterable
                     |
                 Collection
      +--------------+--------------+
      |              |              |
    List           Set           Queue
```

```
            |             |             |
      +-----+-------+   +--+------+     +-+----------+
ArrayList LinkedList  HashSet LinkedHashSet  Deque  PriorityQueue
Vector   Stack       TreeSet(SortedSet)      |
                                        ArrayDeque
                                        LinkedList

-- separately --
                     Map (not part of Collection)
           +-------------+-------------+
       HashMap      LinkedHashMap      TreeMap
```

**Key Point:** `Map` is **not** a child of `Collection` interface — it's a separate hierarchy that stores key-value pairs.

---

### 2.2 Iterable Interface

- `Iterable<T>` is the **root interface** of the Collection framework.
- Any class implementing `Iterable` can be used in a **for-each loop**.
- Declares one key method: `Iterator<T> iterator()`

```java
public interface Iterable<T> {
    Iterator<T> iterator();
}
```

**One-liner:** *"Iterable is the topmost interface — implementing it lets an object be used in an enhanced for-each loop."*

---

### 2.3 Common Operations in All Collection Classes/Interfaces

Since all collections implement the `Collection` interface, they share these common methods:

| Method | Purpose |
|---|---|
| `add(E e)` | Adds an element |
| `remove(Object o)` | Removes a specific element |
| `size()` | Returns number of elements |
| `isEmpty()` | Checks if collection is empty |
| `contains(Object o)` | Checks if element exists |
| `clear()` | Removes all elements |
| `iterator()` | Returns an Iterator to traverse elements |
| `addAll(Collection c)` | Adds all elements from another collection |
| `removeAll(Collection c)` | Removes all matching elements |
| `retainAll(Collection c)` | Keeps only elements present in both collections |
| `toArray()` | Converts collection to an array |

```java
List<String> names = new ArrayList<>();
names.add("Robin");
names.add("Alex");
System.out.println(names.size());       // 2
System.out.println(names.contains("Alex")); // true
names.remove("Alex");
```

---

## 2.4 Iteration on Collection Framework — Different Ways to Iterate

**1) Using `Iterator`**

```
List<String> list = List.of("A", "B", "C");
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

**2) Using `ListIterator` (List only — supports bidirectional traversal)**

```
List<String> list = new ArrayList<>(List.of("A", "B", "C"));
ListIterator<String> lit = list.listIterator();
while (lit.hasNext()) {
    System.out.println(lit.next());
}
while (lit.hasPrevious()) {
    System.out.println(lit.previous());
}
```

**3) Enhanced for-each loop**

```
for (String s : list) {
    System.out.println(s);
}
```

**4) Using `forEach()` with Lambda (Java 8+)**

```
list.forEach(s -> System.out.println(s));
```

**5) Using Stream API (Java 8+)**

```
list.stream().forEach(System.out::println);
```

**6) Classic `for` loop (index-based — only for List, using `get(index)`)**

```
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}
```

**Comparison:**

| Method | Can Modify While Iterating? | Works On |
|---|---|---|
| `Iterator` | Yes, via `it.remove()` | All Collections |
| `ListIterator` | Yes, add/set/remove, bidirectional | List only |
| for-each | No (throws `ConcurrentModificationException`) | All Collections |
| `forEach()` (lambda) | No | All Collections |
| Classic `for` (index) | Yes (with care) | List only |

**One-liner:** *"Use Iterator/ListIterator when you need to modify the collection while iterating; use for-each or forEach() for simple read-only*

*traversal."*

---

## 2.5 List Interface

- **Ordered** collection (maintains insertion order).
- Allows **duplicate elements**.
- Elements accessed via **index** (like an array, but resizable).

**ArrayList**

- Backed by a **dynamic (resizable) array**.
- **Fast random access** ( `get(index)` is O(1)).
- **Slow insertion/deletion** in the middle (elements need to shift) — O(n).

```
List<String> arrayList = new ArrayList<>();
arrayList.add("A");
arrayList.add("B");
arrayList.add(0, "Z");   // insert at index 0 - shifts everything right
System.out.println(arrayList);   // [Z, A, B]
```

**LinkedList**

- Backed by a **doubly linked list** (nodes with next/previous pointers).
- **Fast insertion/deletion** at beginning/middle/end — O(1) (once position is found).
- **Slow random access** ( `get(index)` is O(n) — must traverse from start or end).
- Also implements `Deque` — can be used as a stack or queue too.

```
List<String> linkedList = new LinkedList<>();
linkedList.add("A");
linkedList.add("B");
((LinkedList<String>) linkedList).addFirst("Z");
System.out.println(linkedList);   // [Z, A, B]
```

**Difference between ArrayList and LinkedList**

| Aspect | ArrayList | LinkedList |
|---|---|---|
| **Internal Structure** | Dynamic array | Doubly linked list |
| **Access Time ( `get` )** | O(1) — fast | O(n) — slow |
| **Insertion/Deletion (middle)** | O(n) — slow (shifting) | O(1) — fast (if node reference known) |
| **Insertion/Deletion (at end)** | O(1) amortized | O(1) |
| **Memory Usage** | Less overhead (just array) | More overhead (stores next/prev pointers per node) |
| **Implements** | `List` , `RandomAccess` | `List` , `Deque` |
| **Best Use Case** | Frequent read/search operations | Frequent insertions/deletions |

**One-liner:** *"ArrayList is better for frequent read access; LinkedList is better for frequent insertions/deletions since it avoids shifting elements."*

---

## 2.6 Set Interface

- Does **NOT allow duplicate elements**.
- Mainly used when you need **uniqueness**.

**HashSet**

- Backed by a `HashMap` internally.
- **No guaranteed order** of elements.
- Allows **one null** element.
- O(1) average time for add/remove/contains.

```
Set<String> hashSet = new HashSet<>();
hashSet.add("Banana");
hashSet.add("Apple");
hashSet.add("Apple");     // duplicate - ignored
System.out.println(hashSet);  // order not guaranteed, e.g. [Banana, Apple]
```

**LinkedHashSet**

- Extends `HashSet` but maintains **insertion order** using a linked list internally.
- Slightly slower than `HashSet` due to extra linked list maintenance.

```
Set<String> linkedHashSet = new LinkedHashSet<>();
linkedHashSet.add("Banana");
linkedHashSet.add("Apple");
System.out.println(linkedHashSet);  // [Banana, Apple] - insertion order preserved
```

**TreeSet**

- Implements `SortedSet` — elements stored in **sorted (natural) order**.
- Backed by a **Red-Black Tree** (self-balancing BST).
- Does **not allow null** (throws `NullPointerException`).
- O(log n) for add/remove/contains.

```
Set<Integer> treeSet = new TreeSet<>();
treeSet.add(50);
treeSet.add(10);
treeSet.add(30);
System.out.println(treeSet);  // [10, 30, 50] - sorted automatically
```

**Comparison:**

| | HashSet | LinkedHashSet | TreeSet |
|---|---|---|---|
| **Order** | No order guaranteed | Insertion order | Sorted order |
| **Null allowed** | One null | One null | No null |
| **Backing structure** | HashMap | HashMap + LinkedList | Red-Black Tree |
| **Performance** | O(1) average | O(1) average | O(log n) |

---

## 2.7 Queue Interface

- Follows **FIFO (First In First Out)** principle (mostly), used to hold elements prior to processing.
- Key methods: `offer()` (add), `poll()` (remove & return head, or null if empty), `peek()` (view head without removing).

**Implementing Classes of Queue**

**1) LinkedList** - Can be used as a Queue (implements `Deque`, which extends `Queue`).

```
Queue<String> queue = new LinkedList<>();
queue.offer("A");
```

```
queue.offer("B");
System.out.println(queue.poll());   // A (removed)
System.out.println(queue.peek());   // B (just viewed)
```

**2) PriorityQueue** - Elements are ordered based on **natural ordering** or a provided `Comparator` — **not FIFO**. - The **smallest** (or highest priority) element is always at the head. - Backed by a **binary heap**.

```
Queue<Integer> pq = new PriorityQueue<>();
pq.offer(30);
pq.offer(10);
pq.offer(20);
System.out.println(pq.poll());   // 10 (smallest first)
System.out.println(pq.poll());   // 20
```

**3) ArrayBlockingQueue** - A **bounded, thread-safe** queue backed by an array. - Used in **multi-threaded producer-consumer** scenarios. - If the queue is full, `put()` blocks until space is available; if empty, `take()` blocks until an element is available.

```
BlockingQueue<Integer> bq = new ArrayBlockingQueue<>(5);   // capacity = 5
bq.put(1);     // blocks if queue is full
int val = bq.take();   // blocks if queue is empty
```

**Deque Interface (Double-Ended Queue)**

- Allows insertion/removal from **both ends** (front and rear).
- Can act as both a **Queue** (FIFO) and a **Stack** (LIFO).
- Implementations: `ArrayDeque`, `LinkedList`.

```
Deque<String> deque = new ArrayDeque<>();
deque.addFirst("A");     // insert at front
deque.addLast("B");      // insert at rear
deque.removeFirst();     // remove from front
deque.removeLast();      // remove from rear

// Used as a Stack:
deque.push("X");   // adds to front
deque.pop();        // removes from front
```

**One-liner:** *"Deque supports insertion/removal from both ends, making it useful as both a Queue and a Stack — ArrayDeque is generally preferred over the legacy Stack class."*

---

## 3. Functional Interfaces

- An interface with **exactly one abstract method** (can have multiple default/static methods).
- Also called **SAM (Single Abstract Method)** interfaces.
- Enables the use of **Lambda Expressions**.
- Annotated with `@FunctionalInterface` (optional but recommended — compiler enforces the single-abstract-method rule).

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);   // single abstract method
}

Calculator addition = (a, b) -> a + b;
System.out.println(addition.calculate(5, 3));   // 8
```

### Common Built-in Functional Interfaces (java.util.function package)

| Interface | Method | Purpose |
|---|---|---|

| | | |
|---|---|---|
| `Function<T, R>` | `R apply(T t)` | Takes input, returns output (transformation) |
| `Predicate<T>` | `boolean test(T t)` | Takes input, returns boolean (condition check) |
| `Consumer<T>` | `void accept(T t)` | Takes input, returns nothing (performs action) |
| `Supplier<T>` | `T get()` | Takes no input, returns a value |
| `BiFunction<T, U, R>` | `R apply(T t, U u)` | Takes two inputs, returns output |
| `UnaryOperator<T>` | `T apply(T t)` | Takes and returns the same type |
| `BinaryOperator<T>` | `T apply(T t1, T t2)` | Takes two of the same type, returns same type |

```
Function<Integer, Integer> square = x -> x * x;
System.out.println(square.apply(5));    // 25

Predicate<Integer> isEven = x -> x % 2 == 0;
System.out.println(isEven.test(4));     // true

Consumer<String> printer = s -> System.out.println("Value: " + s);
printer.accept("Hello");                // Value: Hello

Supplier<String> greeting = () -> "Hello World";
System.out.println(greeting.get());     // Hello World
```

**One-liner:** *"A Functional Interface has exactly one abstract method, enabling it to be implemented concisely using a Lambda Expression."*

---

## 4. Lambda Expressions

- A **short block of code** which takes parameters and returns a value — essentially an **anonymous function**.
- Introduced in **Java 8** to enable **functional programming**.
- Used to provide the implementation of a **Functional Interface**.

### Syntax

```
(parameters) -> expression
(parameters) -> { statements; }
```

### Before Lambda (Anonymous Class)

```
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Running...");
    }
};
```

### With Lambda

```
Runnable r = () -> System.out.println("Running...");
```

### Examples

```
// No parameters
```

```java
Runnable task = () -> System.out.println("Task running");

// Single parameter (parentheses optional)
Consumer<String> print = s -> System.out.println(s);

// Multiple parameters
Comparator<Integer> comp = (a, b) -> a - b;

// Multiple statements - needs braces and explicit return
Function<Integer, Integer> factorial = (n) -> {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
};
```

### Lambda with Collections

```java
List<Integer> numbers = List.of(5, 3, 8, 1);

// Sorting using lambda
List<Integer> sorted = new ArrayList<>(numbers);
sorted.sort((a, b) -> a - b);

// forEach with lambda
numbers.forEach(n -> System.out.println(n));

// filtering with Stream + lambda
numbers.stream()
       .filter(n -> n > 3)
       .forEach(System.out::println);
```

### Method Reference (shorthand for simple lambdas)

```java
numbers.forEach(System.out::println);   // instead of n -> System.out.println(n)
```

| Type | Example |
|---|---|
| Static method reference | `ClassName::staticMethod` |
| Instance method reference (particular object) | `object::instanceMethod` |
| Instance method reference (arbitrary object of a type) | `ClassName::instanceMethod` |
| Constructor reference | `ClassName::new` |

### Variable Capture in Lambda

- Lambdas can access **effectively final** local variables from the enclosing scope (value cannot change after being assigned).

```java
int base = 10;   // effectively final
Function<Integer, Integer> addBase = x -> x + base;
System.out.println(addBase.apply(5));   // 15
// base = 20;    // if uncommented -> COMPILE ERROR, base must stay effectively final
```

**Benefits of Lambda Expressions:** - **Less boilerplate code** compared to anonymous classes - Enables **functional-style programming** (works well with Streams) - Improves **readability** for simple behavior passing

**One-liner:** *"Lambda expressions provide a concise way to implement a functional interface's single method, reducing boilerplate compared to anonymous inner classes."*

---

## Quick Revision Summary

- **Generics** → compile-time type safety, no casting; `<T>`, `<? extends T>`, `<? super T>`
- **Collection Framework** → `Iterable` -> `Collection` -> `List` / `Set` / `Queue`; `Map` is separate
- **Iterable** → root interface, provides `iterator()` for for-each support
- **Common operations** → `add`, `remove`, `size`, `contains`, `clear`, `iterator`, etc.
- **Iteration ways** → Iterator, ListIterator, for-each, forEach(lambda), Stream, classic for-loop
- **List** → ordered, allows duplicates; `ArrayList` (fast access, array-backed) vs `LinkedList` (fast insert/delete, node-backed)
- **Set** → no duplicates; `HashSet` (no order), `LinkedHashSet` (insertion order), `TreeSet` (sorted order)
- **Queue** → FIFO; `LinkedList`, `PriorityQueue` (priority order, not FIFO), `ArrayBlockingQueue` (bounded, thread-safe)
- **Deque** → double-ended, works as both Queue and Stack; `ArrayDeque` preferred
- **Functional Interface** → exactly one abstract method; `Function`, `Predicate`, `Consumer`, `Supplier`, etc.
- **Lambda Expression** → concise anonymous function implementing a functional interface; `(params) -> expression/block`
- **Method Reference** → shorthand for simple lambdas ( `ClassName::method` )
- **Effectively Final** → local variables used in lambdas must not change after assignment